



Redes Neurais Artificiais

Prof. Jasmine Araújo
jasmine@ufpa.br

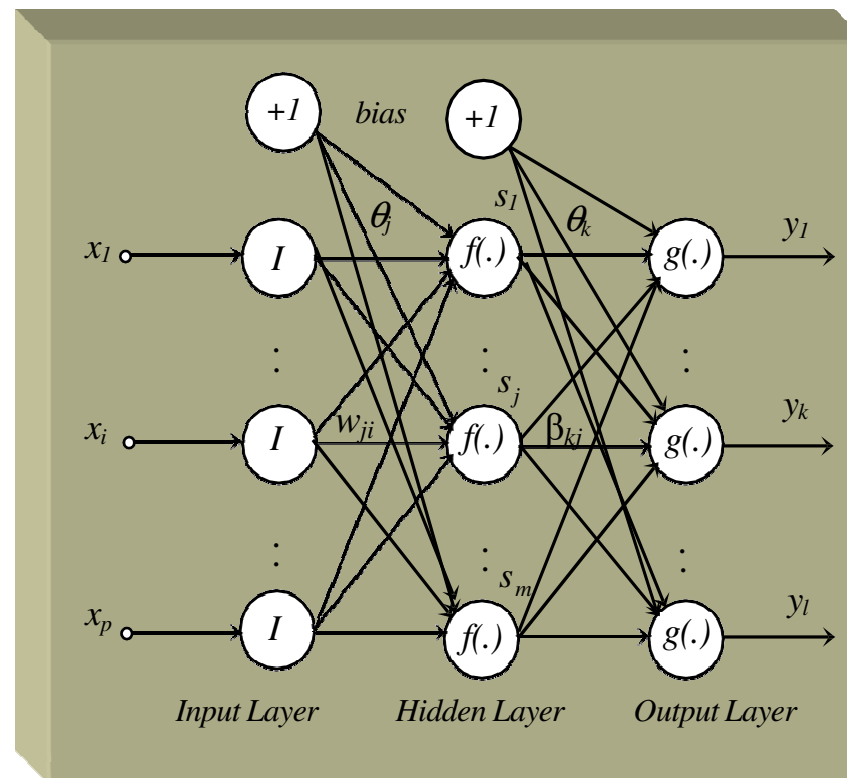


Perceptron Multicamadas

- Multilayer Perceptron ou Perceptron multicamadas foi desenvolvido para lidar com a limitação da aplicação em problemas não lineares.
- É uma rede neural onde o mapeamento entre entradas e saídas não é linear.
- Um Perceptron multicamadas tem camadas de entrada e saída e uma ou mais camadas ocultas com muitos neurônios empilhados juntos.
- E enquanto no Perceptron o neurônio deve ter uma função de ativação que impõe um limite, como ReLU ou sigmóide, os neurônios em um Perceptron Multicamadas podem usar qualquer função de ativação arbitrária.



MLP - Arquitetura Básica





Perceptron Multicamadas - Função Ativação

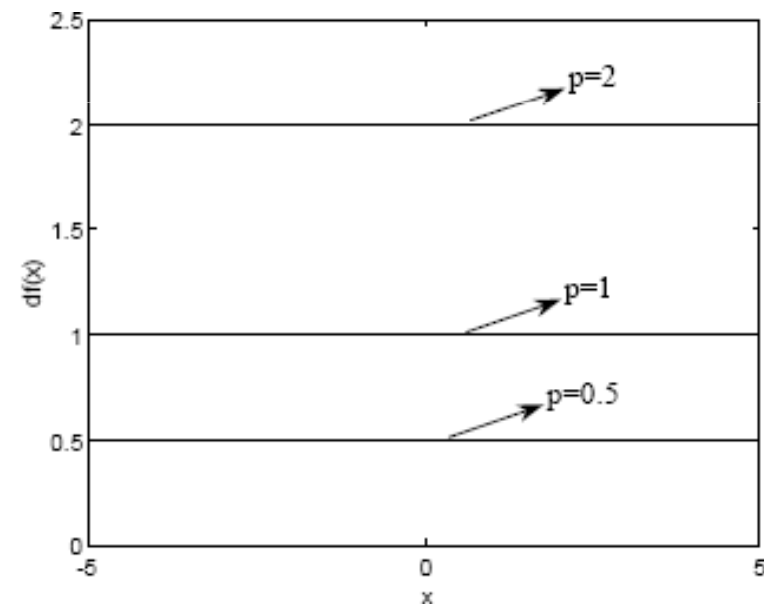
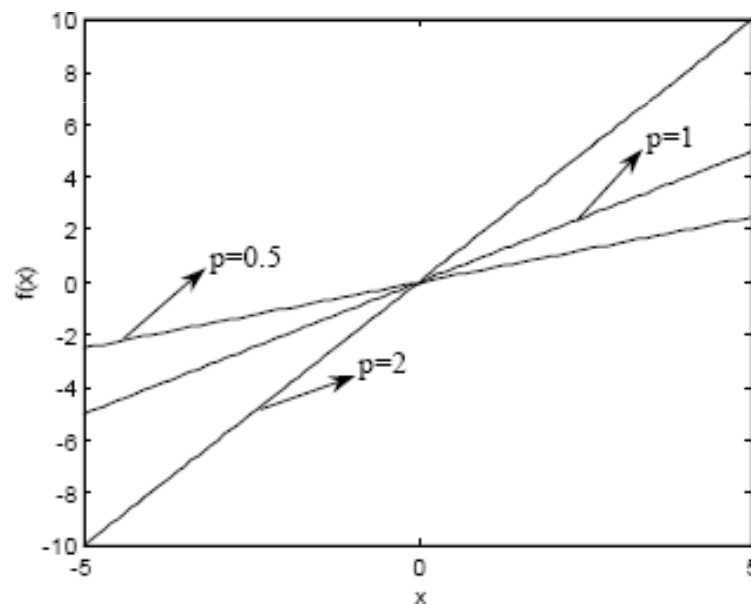
➤ ***Funções mais usadas:***

- ***Linear***
- ***Logística ou Sigmóide***
- ***Tangente hiperbólica***



Função Linear

$$f(x) = px$$

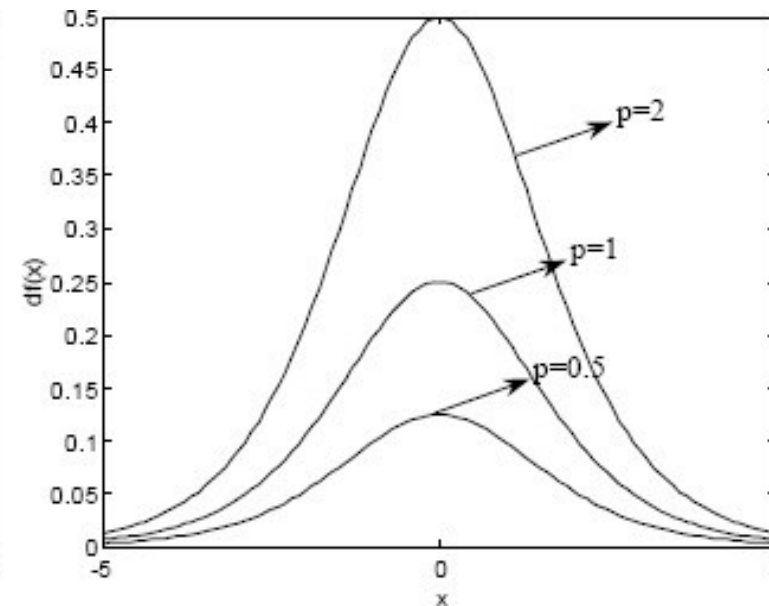
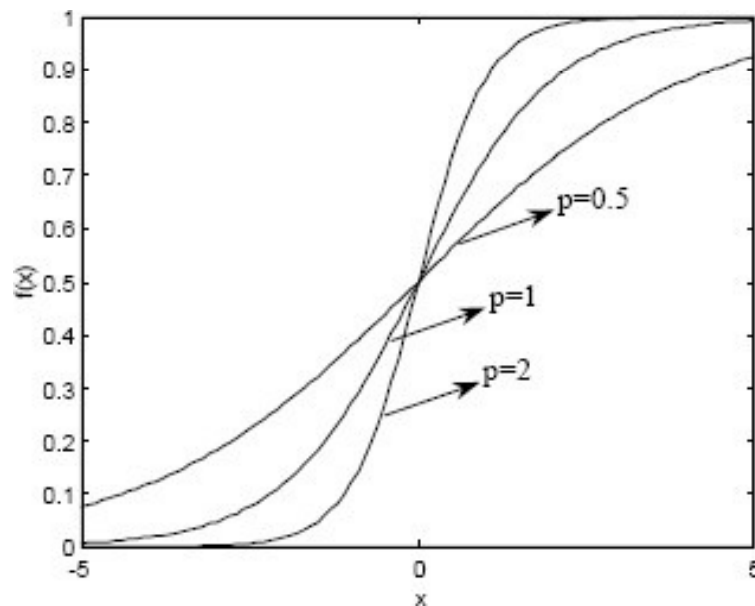


Função Linear e a sua derivada



Função Sigmóide

$$f(x) = \frac{1}{1 + e^{-px}}$$

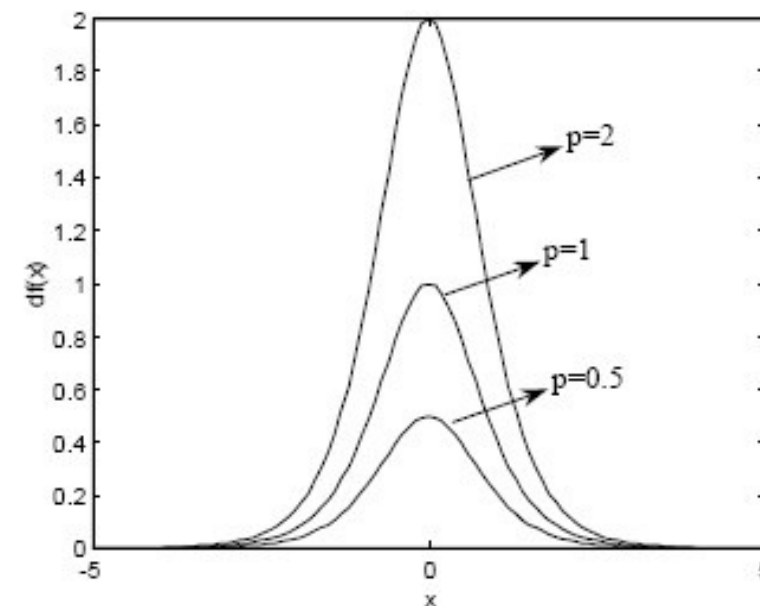
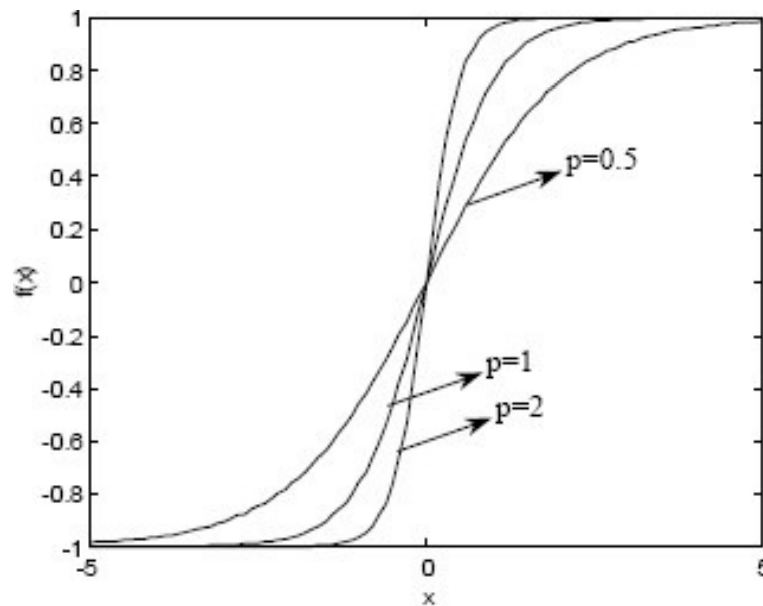


Função sigmóide e a sua derivada



Função Tangente Hiperbólica

$$f(x) = \frac{1 - e^{-px}}{1 + e^{-px}}$$

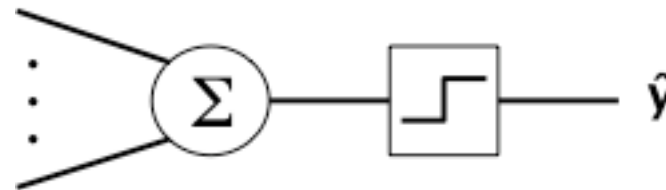


Função Tangente hiperbólica e a sua derivada

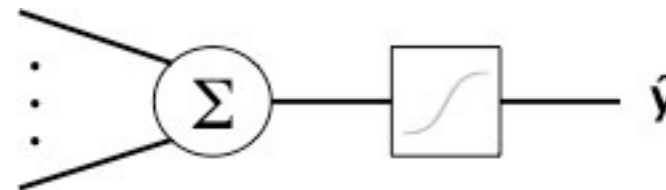


Rede Neural Artificial

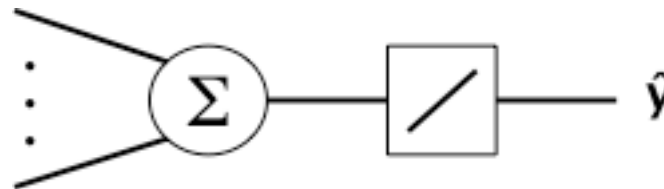
◆ Gate binária



◆ Probabilidade



◆ Regressão





Rede Neural Artificial

- O Perceptron multicamadas se enquadra na categoria de algoritmos feedforward, porque as entradas são combinadas com os pesos iniciais em uma soma ponderada e sujeitas à função de ativação, assim como no Perceptron.
- Mas a diferença é que cada combinação linear é propagada para a próxima camada. Cada camada está alimentando a próxima com o resultado de sua computação, sua representação interna dos dados.
- Isso percorre todo o caminho através das camadas ocultas para a camada de saída.
- Se o algoritmo apenas calculasse as somas ponderadas em cada neurônio, propagasse os resultados para a camada de saída, não seria capaz de aprender os pesos que minimizam a função de custo. Se o algoritmo computasse apenas uma iteração, não haveria aprendizado real.
- Então, definiu-se o algoritmo de **backpropagation** para ajustar os pesos na rede com o objetivo de minimizar a função custo.



Backpropagation Treinamento

♦♦ O Funcionamento do **backpropagation**

- Após atualizar todos os pesos da rede, o processo recomeça. Um ciclo completo é chamado de época
- Quando uma época é completada, a tendência é que o erro quadrático médio diminua, até alcançar um mínimo pré-estabelecido, momento no qual a rede converge.

É processado através da minimização da soma dos erros médios quadráticos sobre um conjunto de treinamento.



Backpropagation Treinamento

♦♦ O Funcionamento do **backpropagation**

- Após atualizar todos os pesos da rede, o processo recomeça. Um ciclo completo é chamado de época
- Quando uma época é completada, a tendência é que o erro quadrático médio diminua, até alcançar um mínimo pré-estabelecido, momento no qual a rede converge.

É processado através da minimização da soma dos erros médios quadráticos sobre um conjunto de treinamento.



Backpropagation Treinamento

- ❖ O sinal de erro no neurônio de saída j no instante i (apresentação de um padrão) é calculado por:

$$e_j(i) = d_j(i) - y_j(i)$$

- ❖ Considerando todos os neurônios de saída da RNA, a soma dos erros quadráticos, sobre todas as saídas j , para o padrão de treinamento i , é dada por:

$$E(i) = \frac{1}{n} \sum_j e_j^2(i)$$

- ❖ Por sua vez, se formos considerar todos os N padrões apresentados para RNA, a função custo que deve ser minimizada durante o processo de aprendizagem corresponde ao erro quadrático médio, definido como:

$$E = \frac{1}{N} \sum_i \sum_j e_j^2(i)$$



Atualização dos pesos

◆ Batch updates

- Atualização dos pesos são realizadas após o passado todo o conjunto de dados de treinamento

◆ Pattern updates

- Atualizações dos pesos são realizadas após cada vetor de entrada ser apresentado



Backpropagation Treinamento

❖ Taxa de Aprendizado:

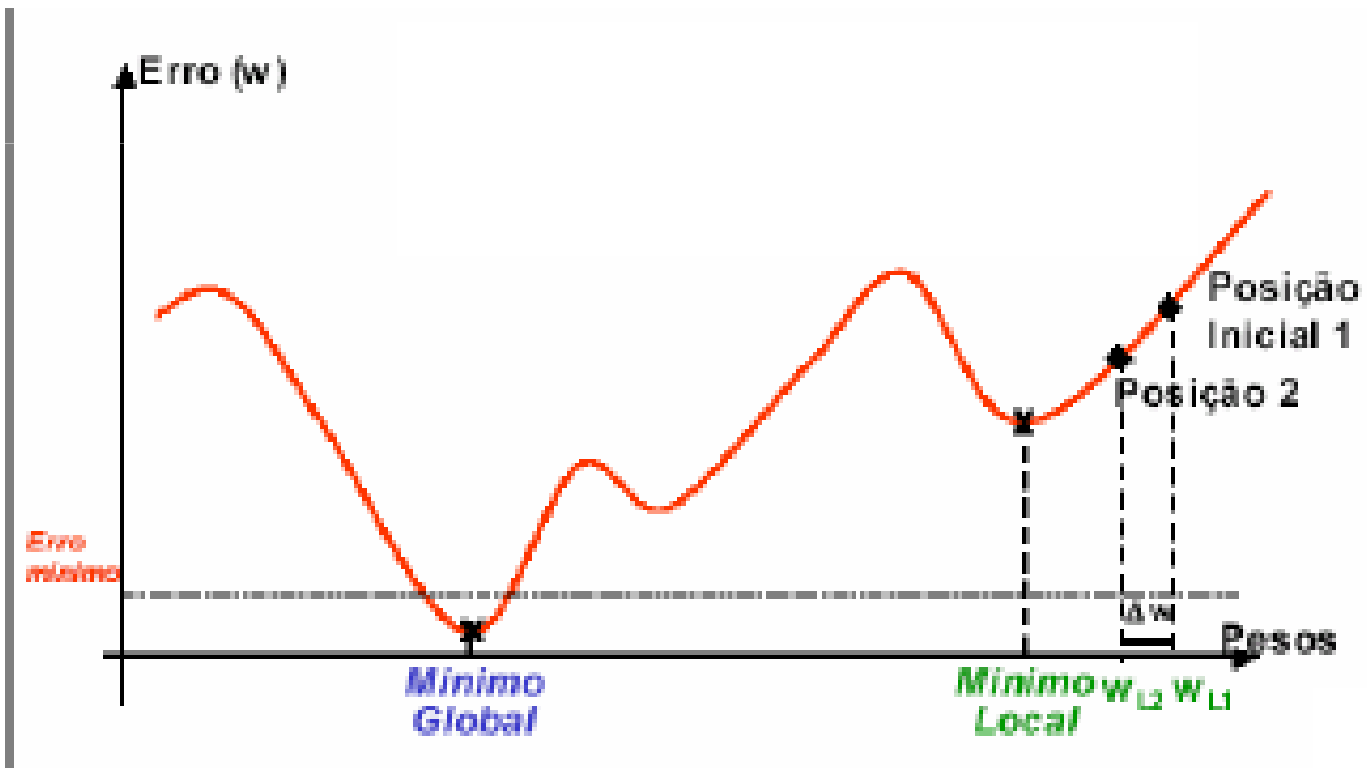
- Quanto menor for o valor da taxa de aprendizado, menor serão as variações dos pesos sinápticos de uma iteração para outra, e mais suave será a trajetória no espaço de pesos – aprendizado lento
- Quanto maior for o valor da taxa de aprendizado, maior serão as variações dos pesos sinápticos de uma iteração para outra, podendo tornar a rede instável



Backpropagation Treinamento

Inicialização dos Pesos

♦ *Problema do mínimo local.*





Backpropagation Treinamento

Critérios de Parada:

♦ Pelo número de ciclos de treinamento

- treinamento termina após a execução de um número determinado de ciclos de treinamento.

♦ Pelo valor do erro

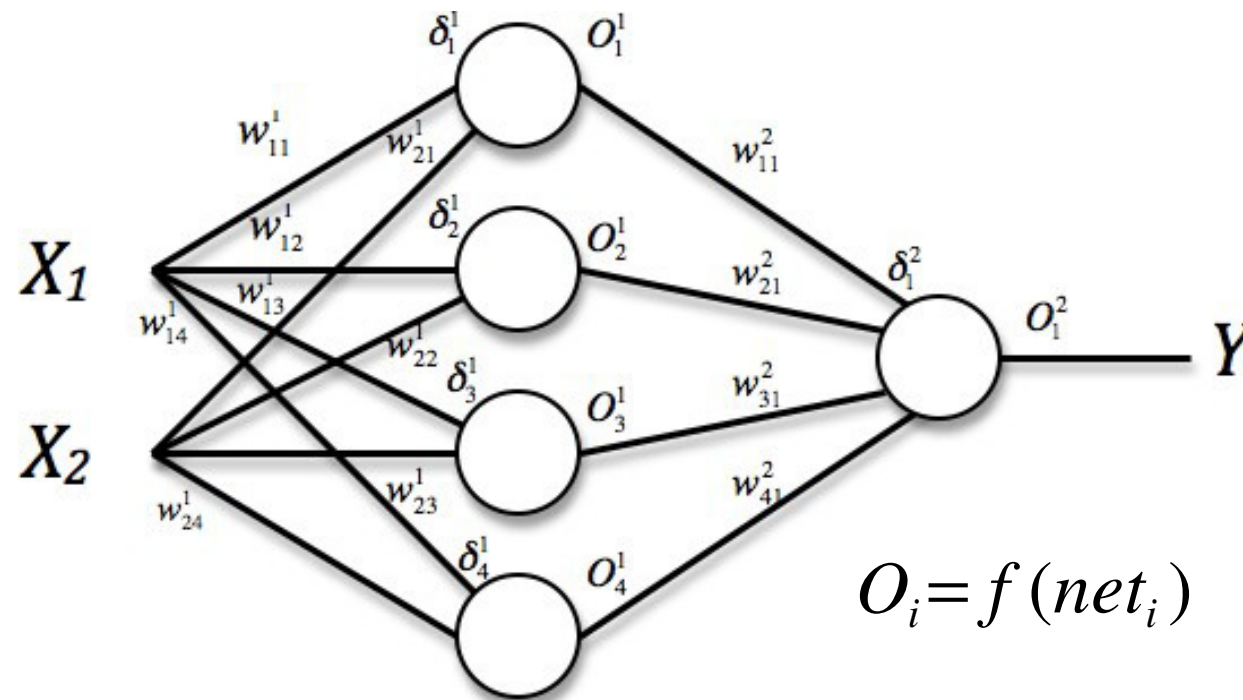
- o treinamento termina quando valor do erro é menor ou igual a valor predeterminado.

♦ Pelo critério de validação

- Treinamento termina quando o erro no conjunto de validação para de decrescer



Atualização por retropropagação





Atualização por retropropagação

$$\otimes w_{ji} = \eta e_j f'(net_j) O_i = \eta \delta_j O_i$$

$$\begin{pmatrix} \text{correção} \\ \text{do peso} \\ \Delta w_{ji}(n) \end{pmatrix} = \begin{pmatrix} \text{Parâmetro} \\ \text{da taxa de} \\ \text{aprendizado } \eta \end{pmatrix} \cdot \begin{pmatrix} \text{Gradiente} \\ \text{local} \\ \delta_j(n) \end{pmatrix} \cdot \begin{pmatrix} \text{sinal de entrada} \\ \text{do neurônio } j \\ O_i(n) \end{pmatrix}$$

O gradiente local depende se o neurônio j é um nó de saída ou um nó oculto:

Se neurônio j é um nó de saída $\delta_j = e_j f'(net_j)$

Se neurônio j é um nó oculto $\delta_j = f'(net_j) \sum_k \delta_k w_{kj}$



Desvantagens

As desvantagens do Multi-layer Perceptron (MLP) incluem:

- MLP com camadas ocultas tem uma função de perda não convexa onde existe mais de um mínimo local. Portanto, inicializações de peso aleatório diferentes podem levar a uma precisão de validação diferente.
- O MLP requer o ajuste de vários hiperparâmetros, como o número de neurônios ocultos, camadas e iterações.



♦Dúvidas???